

Virtualization & Container

Virtualizzazione e Integrazione di Sistemi

Prof. Vittorio Ghini

a.a. 2024-25

Ingegneria e Scienze Informatiche

Università di Bologna

Terminologia della Virtualizzazione

- **Sistema host:** è la macchina fisica su cui gira il sistema operativo principale e che poi ospiterà le macchine virtuali
- **Sistema guest (macchina virtuale):** è l'insieme delle risorse hardware e il sistema operativo che viene eseguito “sopra” il sistema host
- **Hypervisor** o Virtual machine monitor: è il componente software che crea e manda in esecuzione le VM;
 - ha il compito di astrarre e rendere disponibili le risorse hardware virtualizzate;
 - svolge compiti di monitoraggio/sicurezza e viene usato ai fini di debug

Virtualizzazione Desktop

- Consente di usare una VM che esegue sul PC dell'utente, perciò la macchina virtuale sfrutta le periferie della macchina fisica (tastiera, mouse, schermo) per consentire all'utente l'interazione col sistema operativo guest.
- Viene realizzata mediante l'utilizzo di particolari software, e permette di eseguire altri sistemi operativi “sopra” il sistema operativo host.
- Vengono perciò utilizzati hypervisor che vengono classificati come Type2 (hosted)
- La virtualizzazione desktop è utile per far funzionare un software non compatibile con il sistema operativo dell'host o un software che si vuole mantenere separato dal software dell'host.

Principali Software per Virtualizzazione Desktop

- VirtualBox (open source e multiplatforma)
- Qemu/KVM + libvirt + virt-manager (open source, Linux)
- VirtualPC (closed-source, Windows)
- Vmware player (closed-source, Windows, MacOSX)
- UTM (open source, MacOSX)
- Parallels (closed-source, MacOSX)

Virtualizzazione Server

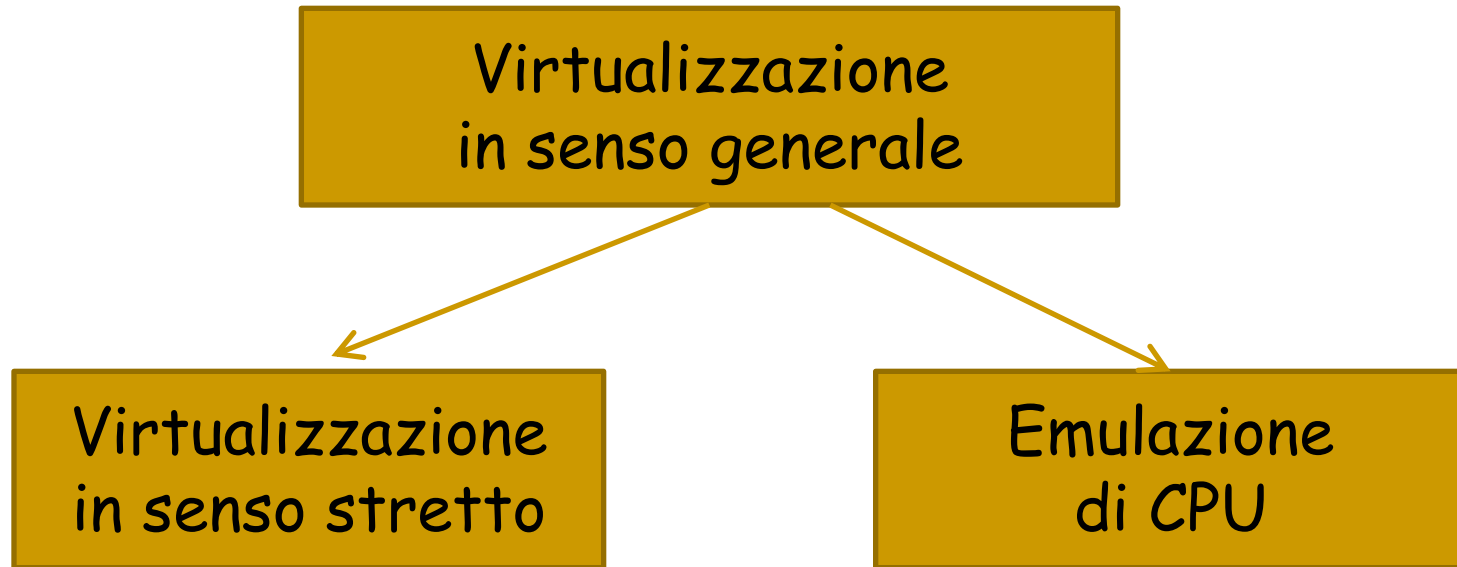
- Esegue su HW in sale macchine, l'utente perciò non opera direttamente sfruttando le periferiche della macchina fisica su cui opera la virtualizzazione (tastiera, mouse, schermo) , bensì si collega da remoto alla macchina virtuale.
- Viene realizzata mediante l'utilizzo di sistemi operativi dedicati o customizzati ad hoc, e permette di eseguire altri sistemi operativi direttamente sull'hardware.
- Viene utilizzata per virtualizzare i server/servizi/appliance di organizzazioni di dimensioni diverse, fino ad ambienti enterprise e datacenter.
- Può essere utilizzata per virtualizzare sistemi operativi desktop, che poi vengono utilizzati dagli utenti attraverso sessioni remote,
- Vengono utilizzati hypervisor che vengono classificati come Type1 (bare-metal).

Principali Software per Virtualizzazione Server

- Proxmox: open source, basato su Debian, sfrutta Qemu/KVM + libvirt + virt-manager.
- Hyper-V: closed source, basato su Windows Server
- VmWare ESX/ESXi: closed source, basato su RedHat
- Xen: open source, basato su *NIX

Virtualizzazione vs. Emulazione

- distinguiamo



Virtualizzazione vs. Emulazione

- Tramite **virtualizzazione** è possibile eseguire uno o più sistemi operativi (ed il relativo software applicativo) da un unico PC, in un ambiente protetto e monitorato che prende il nome di *macchina virtuale (VM)*.
- Il sistema operativo in cui viene eseguita la macchina virtuale, viene detto *ospitante (host)*
- La macchina virtuale è chiamata *ospite (guest)*.
- Il codice della macchina virtuale viene eseguito direttamente dal sistema ospitante, ma il sistema ospite "pensa" di essere eseguito su una macchina reale.
 - ❑ Il codice della macchina virtuale, perciò deve essere codice macchina eseguibile dalla macchina hardware reale sottostante.
 - ❑ Non posso virtualizzare un sistema operativo, che dovrebbe girare su un x86 a 64 bit, su un processore ARM.
- **Emulazione di processore.** In questo caso l'hardware viene completamente emulato dal programma di controllo, cioè ogni istruzione che il sistema guest esegue viene tradotta in una sequenza di istruzioni della macchina host. Il processo di emulazione risulta più lento rispetto alle due forme di virtualizzazione precedenti, a causa della traduzione delle istruzioni dal formato del sistema ospite a quello del sistema ospitante.

Domanda:

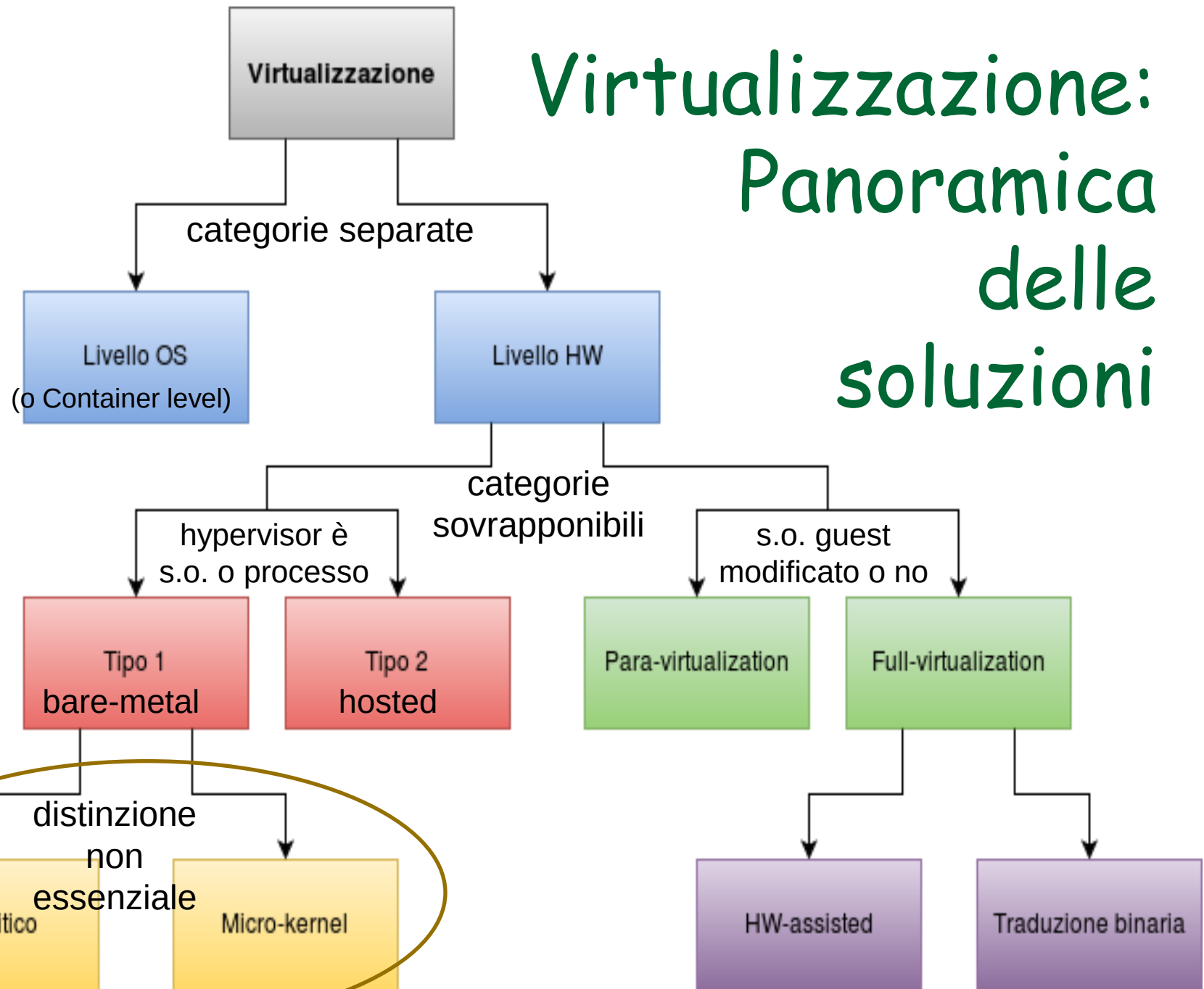
- Ma allora perchè ad esempio, posso installare una macchina virtuale per x86 a 32 bit su un sistema operativo che gira su un processore fisico x86 a 64 bit ?
- Puoi farlo perché un processore x86 a 64 bit mantiene la **retrocompatibilità** con le istruzioni a 32 bit, ovvero è in grado di distinguere le istruzioni a 32 e a 64 bit ed è in grado di eseguirle entrambe.
 - In un certo senso, un processore x86 a 64 bit riesce ad emulare automaticamente un processore x86 a 32 bit.
- Tanto è vero che se fate il contrario, cioè cercare di installare una macchina virtuale x86 a 64 bit su un processore fisico x86 a 32 bit, non ce la fate (caso capitato in laboratorio di sist.op. sul portatile a 32 bit di qualcuno).

Esempio di Emulatore di Processore

QEMU

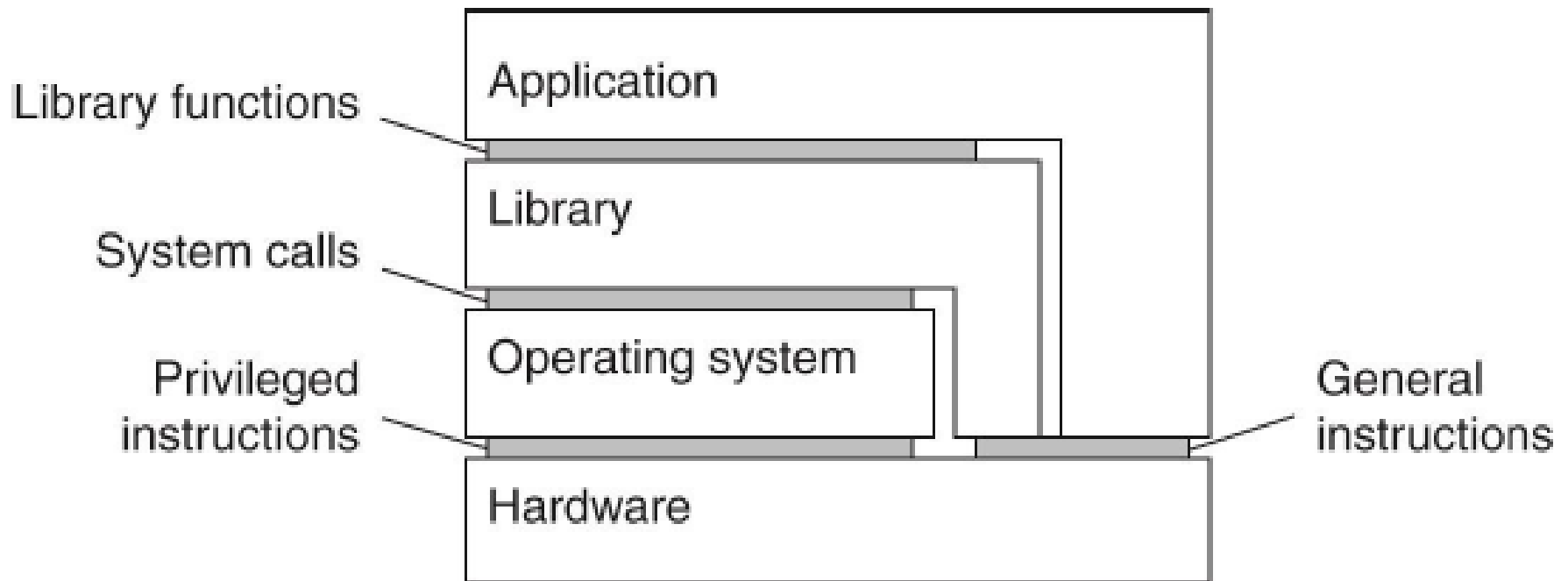
- QEMU è un emulatore di processore.
- Emula l'hardware del sistema ospitante esaminando dinamicamente il codice eseguito all'interno della macchina virtuale e traducendolo in istruzioni comprensibili alla macchina ospitante.
- QEMU è in grado di emulare numerose architetture hardware, tra cui x86, x86_64, ARM, SPARC, PowerPC, e MIPS.
- Il sistema ospitante è però deve essere una delle seguenti architetture:
x86, x86_64 e PowerPC
- Come curiosità e per capire le prestazioni:
 - Nel caso di architetture x86 esiste una implementazione apposita (detta acceleratore (kqemu)) che limita la traduzione dinamica delle istruzioni, permettendo di raggiungere prestazioni attorno al 30-50% di quelle del sistema ospitante (con una emulazione non accelerata le prestazioni calano al solo 5-10%).

Virtualizzazione: Panoramica delle soluzioni



Spazio Kernel e Spazio Utente

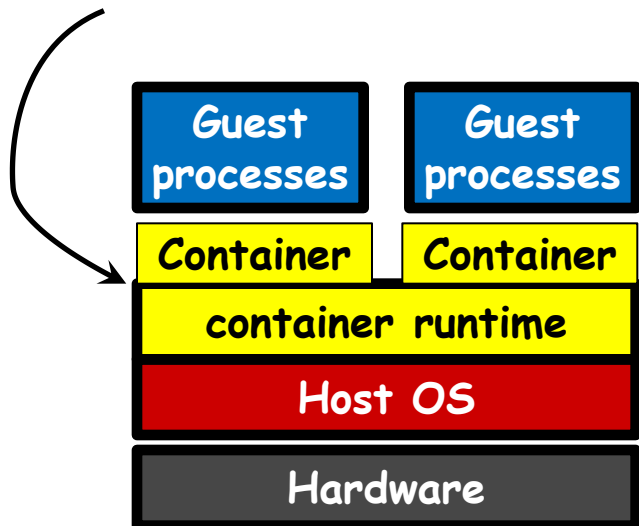
- Le moderne CPU prevedono confini di sicurezza (ring).
- Le istruzioni macchina (fornite dal livello ISA) si distinguono in Istruzioni Privilegiate e Istruzioni Generali
- Le istruzioni Privilegiate possono essere eseguite solo dal kernel del sistema operativo, entrando in un apposito Ring di esecuzione della CPU.
- Le istruzioni Generali possono essere eseguite anche dallo spazio utente



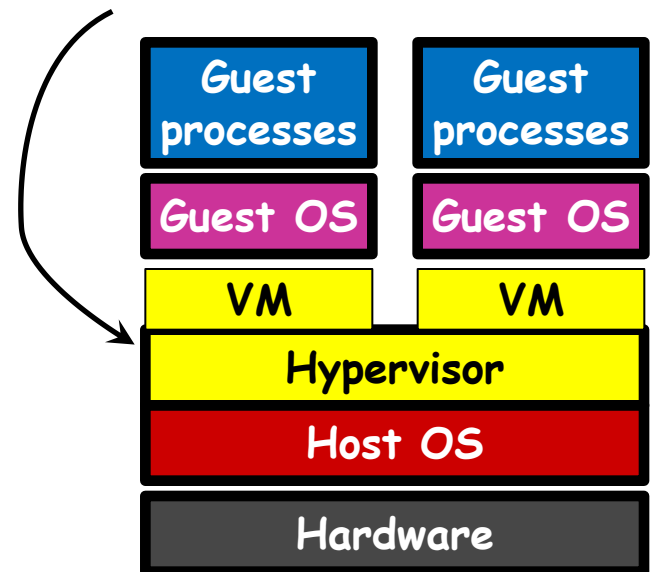
Virtualizz. del Livello HW o Livello OS

- La prima grande distinzione riguarda il tipo di risorse virtuali che si vuole presentare all'utente: **macchine virtuali** oppure **partizioni (container) isolate**.
- Nel caso delle **macchine virtuali** (livello Hardware) all'utente del sistema di virtualizzazione viene presentata un'interfaccia su cui installare un sistema operativo, quindi una CPU virtuale (ma dello stesso tipo della CPU fisica) e risorse HW virtuali.
- Nel caso dei **container** (livello OS) all'utente viene presentata una partizione (container) del sistema operativo corrente, su cui installare ed eseguire applicazioni che rimangono isolate nella partizione, pur accedendo ai servizi di uno stesso o.s.

OS-level virtualization



Hardware virtualization



Livello HW vs. Livello OS

- La virtualizzazione OS-level è composta da un solo kernel (quello del sistema operativo host) e multiple istanze isolate di user-space (chiamate anche partizioni o contenitori), che possono essere avviate e spente in maniera indipendente tra loro. Viene garantito l'isolamento del filesystem, IPC e network. Inoltre fornisce un sistema di gestione delle risorse quali CPU, memoria, rete e operazioni I/O.
- Nella virtualizzazione hardware, i sistemi operativi vengono eseguiti in modo concorrente sullo stesso hardware e possono solitamente essere eterogenei. L'**hypervisor** (chiamato anche *Virtual Machine Monitor*) si occupa di multiplexare l'accesso alle risorse hardware e garantire protezione e isolamento tra le macchine.

Quindi:

- **OS-level (o container-level):** offre contenitori per applicazioni ed esegue un solo kernel, quello del o.s. Host.
 - Vantaggi: basso overhead per il context-switch, basso overhead di memoria
 - Svantaggi: non può ospitare sistemi operativi differenti, l'isolamento non può essere del tutto perfetto
- **hardware-level:** quando offre macchine virtuali.
 - Vantaggi/Svantaggi: speculari rispetto a OS-level

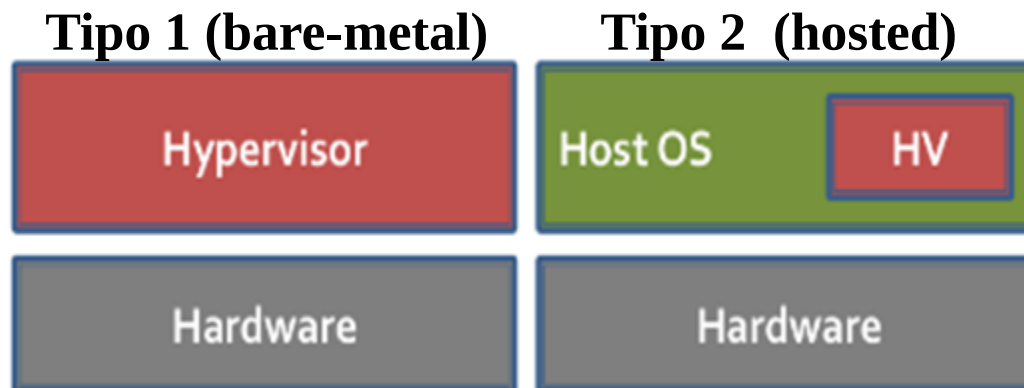
Livello OS (Container) - Docker

- Apriremo più tardi una finestra accennando ad un caso prominente di virtualizzazione a livello di container su sistemi Linux: **Docker**
- Ne parliamo poiché Docker viene spessissimo utilizzato per dispiegare micro-servizi su sistemi in cloud.

Esistenza del S.O. Ospitante

(Collocazione dell'Hypervisor)

- Nella categoria della virtualizzazione hardware (quella che fornisce una CPU virtuale ma dello stesso tipo della CPU fisica) possono esistere casi in cui il sistema operativo appoggiato sulla macchina fisica non esiste.
- Cioè, l'Hypervisor può svolgere il ruolo di sistema operativo essendo appoggiato sulla macchina fisica oppure essere un processo all'interno di un altro s.o.
- Si definisce virtualizzazione di **tipo 1** o **bare-metal** un sistema di virtualizzazione nel quale il sistema operativo *host* è assente e le sue funzioni vengono sostituite dall'hypervisor.
- Si definisce virtualizzazione di **tipo 2** o **hosted** un sistema di virtualizzazione nel quale l'hypervisor è un normale processo utente sul sistema operativo *host* (a meno di specifici moduli da integrare nel kernel).



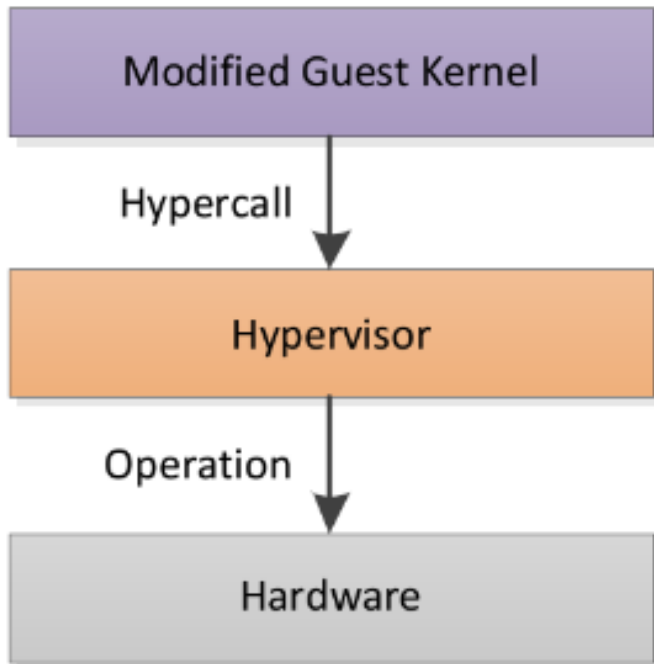
Esistenza del S.O. Ospitante

- Ad esempio, il virtualbox che forse avete installato sul vostro portatile per sistemi operativi è un hypervisor hosted.

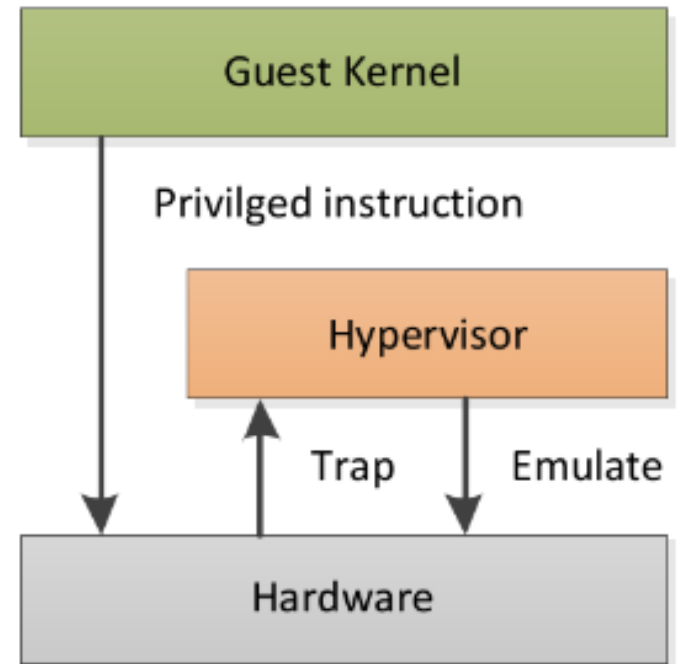
Full e Para Virtualization

- Vi ricordo che, da definizione di virtualizzazione, la virtualizzazione hardware fornisce una astrazione di macchina virtuale (CPU) che però deve essere dello stesso tipo della macchina fisica sottostante.
- La virtualizzazione hardware fornisce una interfaccia, su cui installare un sistema operativo, e può essere distinta in **Para-virtualization** e **Full-virtualization**

Para-virtualization



"Classical" Full-virtualization



Full e Para Virtualization

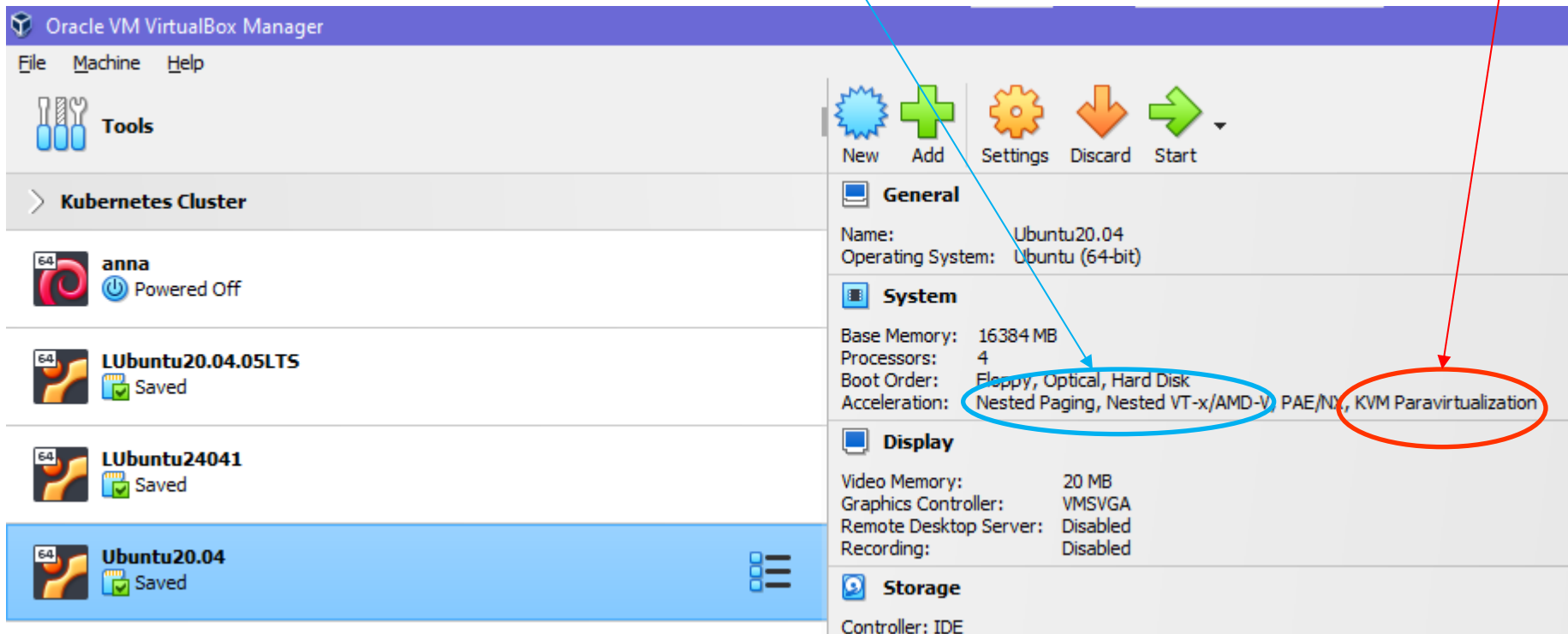
- La virtualizzazione hardware fornisce una macchina virtuale (con CPU di stesso tipo della fisica) sulla cui interfaccia si può installare un sistema operativo.
- Distinguiamo due tipi di virtualizzazione hardware:
- **Para-virtualization:** la macchina virtuale presenta **un'interfaccia differente** confrontata con una macchina fisica. Questo comporta dover modificare il sistema operativo guest per consentire l'esecuzione all'interno della macchina virtuale stessa.
 - L'hypervisor espone un insieme di API che il sistema operativo guest dovrà utilizzare, **in particolare per utilizzare le istruzioni privilegiate**. Le chiamate a queste funzioni sono usualmente definite **Hypercall**.
- **Full-virtualization:** nella full-virtualization (caso oggi molto più comune) fornisce macchine virtuali che hanno **la medesima interfaccia di una macchina fisica**. Idealmente il sistema operativo guest non può identificare che si trova su una macchina virtuale (in realtà di solito è sempre possibile dedurlo attraverso opportune tecniche). Il grande vantaggio della virtualizzazione completa è il **non dover modificare il sistema operativo**.
 - L'hypervisor adotterà un sistema di TRAP per gestire le istruzioni privilegiate.

Para Virtualization

- Nella Para-virtualization si è detto che occorre modificare il kernel del sistema operativo guest affinché possa utilizzare le API dell'hypervisor, quando deve utilizzare le istruzioni privilegiate della CPU.
- **E' vero ma, ad esempio, non pensiate che sia veramente necessario modificare un sistema linux per farlo girare sopra ad un hypervisor tipo Xen!**
- Il kernel di Linux fornisce già dei moduli che, installati ed eseguiti, sostituiscono alcune porzioni di codice (cambia dei puntatori a funzione nel kernel) in modo che, quando dovrebbero essere chiamate delle istruzioni privilegiate, in realtà vengono chiamate delle funzioni messe a disposizione dall'hypervisor.

Para Virtualization - è diffusa

- La tecnica della Para Virtualizzazione è usata da alcuni hypervisor molto diffusi, ad esempio Virtualbox. Lo si capisce guardando la configurazione di una VM creata dentro VirtualBox.
- Si seleziona la VM e compare a destra la sua configurazione: notare come nel riquadro «System» nella riga «Acceleration» si veda scritto «**KVM Paravirtualization**», che è un tipo di paravirtualizzazione. Compaiono però anche alcune tecniche di Full virtualization (**Nested Paging**, vedi slide successive).



Il problema delle istruzioni privilegiate nella Full Virtualization

- Nella Full-virtualization esiste il grosso problema delle istruzioni privilegiate, che dovrebbero essere eseguite in kernel mode (cioè in un ring privilegiato).
- Potenzialmente, queste istruzioni possono svolgere compiti senza alcuna limitazione, e potrebbero travalicare i confini delle macchine virtuali.
- Occorre perciò limitare e controllare le azioni che vengono svolte dalle istruzioni privilegiate, eseguite nel ring 0, cioè in kernel mode.
- Esistono due diverse soluzioni :
 - Traduzione dinamica binaria (**binary translation**)
 - Assistita dall'hardware (**HW assisted**)
- Domanda: Perché lo stesso problema non sussiste nella Para-virtualization?
 - Perché nella Para-virtualization il sistema operativo ospitato non invoca direttamente le istruzioni privilegiate, bensì invoca delle funzioni appositamente scritte dall'hypervisor, che si occupano di non fare danni.

Binary translation nella Full Virtualization

- La traduzione binaria (**binary translation**) consiste nel riscrivere dinamicamente a run-time, man mano che vengono eseguite, le parti del codice del sistema operativo guest che dovrebbero essere eseguite in modo privilegiato.
- La tecnica è stata applicata su larga scala per primo da Vmware.
- Le istruzioni non privilegiate del guest sono eseguite direttamente.
- Per le istruzioni privilegiate, invece, la CPU opera come se fosse in modalità debug, sospendendo via via l'esecuzione delle istruzioni. Il codice viene analizzato e parti di questo codice copiate in una cache e qui in parte sostituito con codice che accede alle risorse virtualizzate invece che alle risorse reali, se necessario.
- Mantenendo parti di codice in una cache, evitiamo di dover riscrivere più volte lo stesso codice quando viene eseguito un loop.

Hardware-assisted nella Full Virtualization (1)

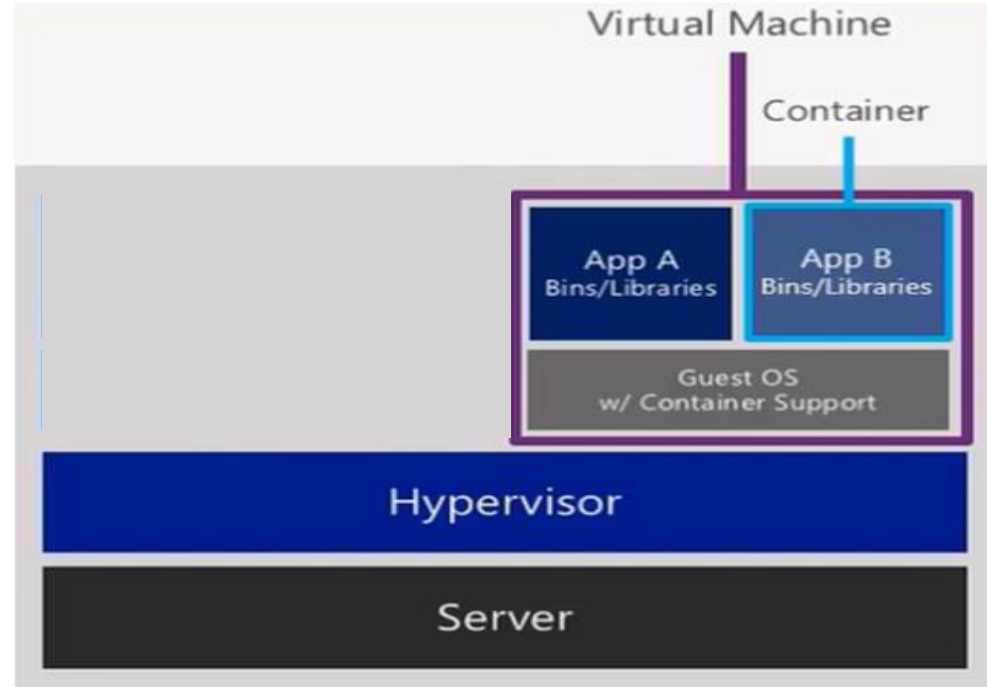
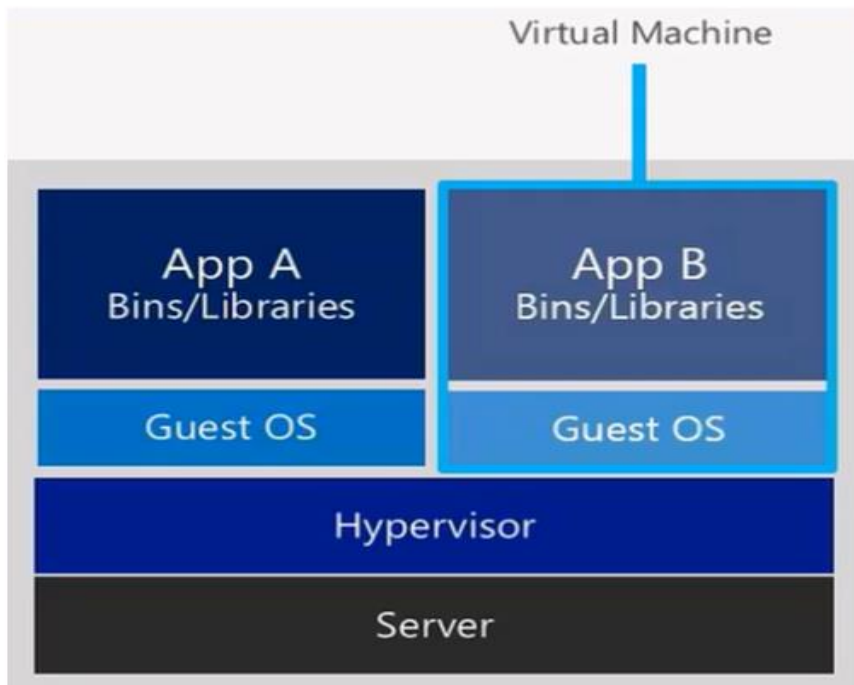
- L'hardware assisted virtualization compare nel 2007 sui processori Intel e AMD. Queste funzionalità sono note come Intel VT-x or AMD-V.
- In sostanza, aggiungere un nuovo ring all'architettura standard x86, chiamato **Ring -1**.
- In questo caso, il sistema operativo host e hypervisor vengono eseguiti nel nuovo ring -1, mentre il sistema operativo guest è eseguito nel Ring 0. Così facendo il sistema operativo guest può eseguire le istruzioni privilegiate, sotto però il controllo dell'hypervisor.
- Infatti, è l'hypervisor che decide quando mettere la CPU in modo -1 oppure in modo 0.
 - Vengono messe a disposizione alcune istruzioni "VMPTRLD, VMPTRST, VMCLEAR, VMREAD, VMWRITE, VMCALL, VMLAUNCH, VMRESUME, VMXOFF, and VMXON. These instructions permit entering and exiting a virtual execution mode where the guest OS perceives itself as running with full privilege (ring 0), but the host OS remains protected."

Hardware-assisted nella Full Virtualization (2)

- Vengono poi aggiunte delle funzionalità che aggiungono un ulteriore livello di mappatura delle pagine in memoria:
 - Intel **Extended Page Tables** (EPT).
 - Una volta individuata la pagina in memoria virtuale nel modo classico, una ulteriore tabella stabilisce dove viene collocata la pagina dall'Hypervisor (e poi si applica il consueto metodo di paginazione su disco).
- Infine, MERAVIGLIOSO, vengono aggiunte delle tabelle con le informazioni su ciascuna macchina virtuale. Prima ne esisteva una sola, quindi non si potevano annidare le VM, una dentro l'altra.
 - **VMCS shadowing**.
 - "The *virtual machine control structure* (VMCS) is a data structure in memory that exists exactly once per VM, while it is managed by the VMM. With every change of the execution context between different VMs, the VMCS is restored for the current VM, defining the state of the VM's virtual processor"

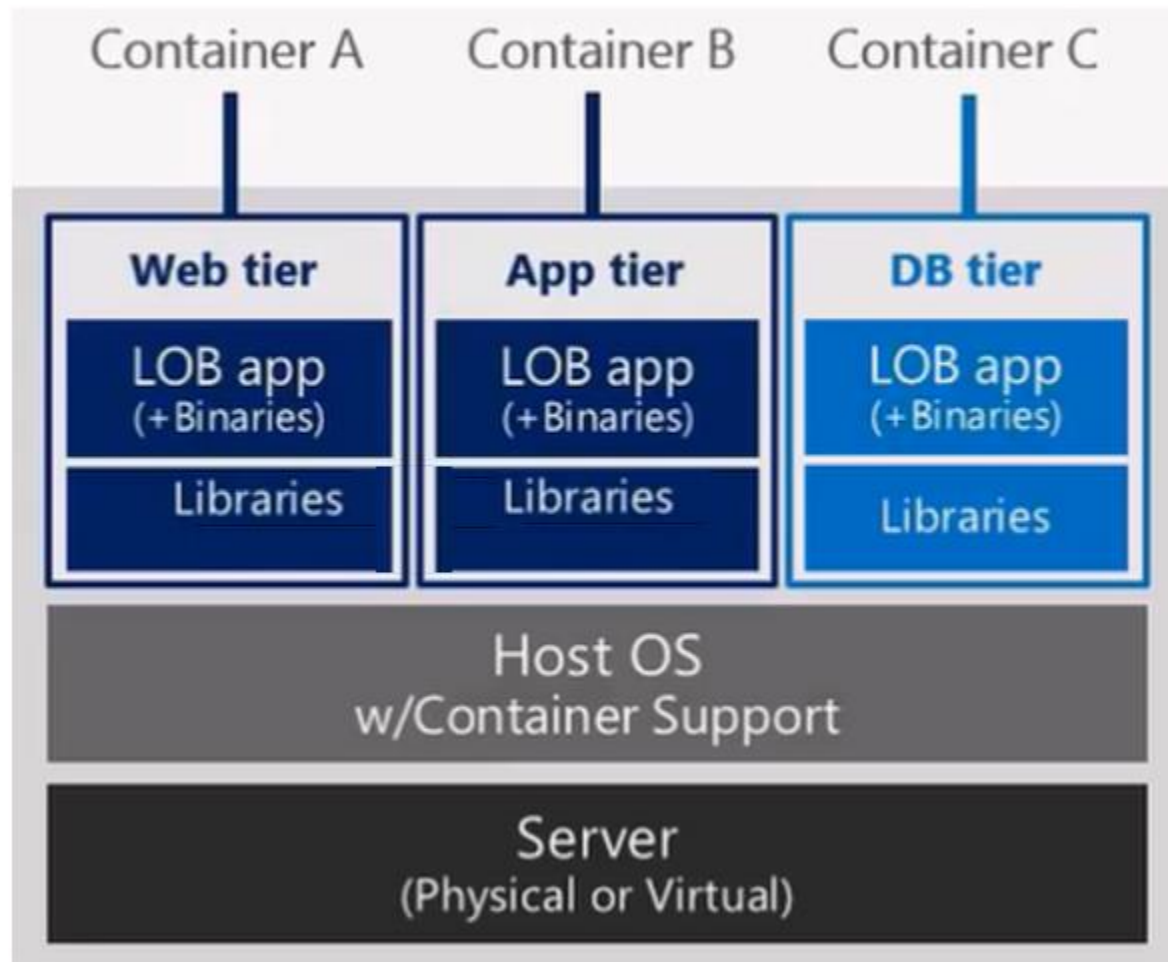
2 parole sui Container - perché?

- Creano uno spazio utente isolato, con proprie librerie e files, isolando una applicazione dal resto del sistema operativo.
 - ❑ **Pacchettizzo una applicazione, rendendola pronta per il deployment.**
- Più container possono appoggiarsi ad uno stesso spazio kernel, risparmiando spazio disco e condividendone i servizi di base.
- Costruito e customizzato un container per una specifica applicazione, posso replicare quel container più volte, anche su una stessa VM o su VM diverse.



2 parole sui Container - perché?

- Può essere comodo salvare i dati permanenti delle applicazioni di un container su un servizio separato, magari anch'esso contenuto in un container.
- Posso costruire applicazioni composte da tanti micro-componenti riutilizzabili, ciascuno dei quali isolato in un container.



Quali Container ?

Ricordiamo solo i principali:

- **Docker** (più precisamente Open Container Initiative OCI specification)
 - Si appoggia sul s.o. **Linux** che fornisce, a livello kernel) un supporto per container detto **LXC (Linux Container)**.
 - Non confondete LXC con LXD che è un sistema per dispiegare facilmente virtual machine Linux.
- **Hyper-V** di Microsoft (fornisce unità di isolamento chiamate Container, in realtà sono delle macchine virtuali).
 - Tra l'altro, si parla di container di Hyper-V, ma su sistemi diversi (ad esempio, windows server e windows 10) sono piuttosto differenti, come tipo di isolamento fornito
- **Container di Windows Server** (sono effettivamente dei container)
- NB: E' possibile installare il s.o. Windows server in una configurazione minimale, detto Nano Server, ottimizzato per stare dentro macchine virtuali di hyper-v o per sostenere container di windows server.
<https://docs.microsoft.com/it-it/windows-server/get-started/getting-started-with-nano-server>

Windows Nano Server

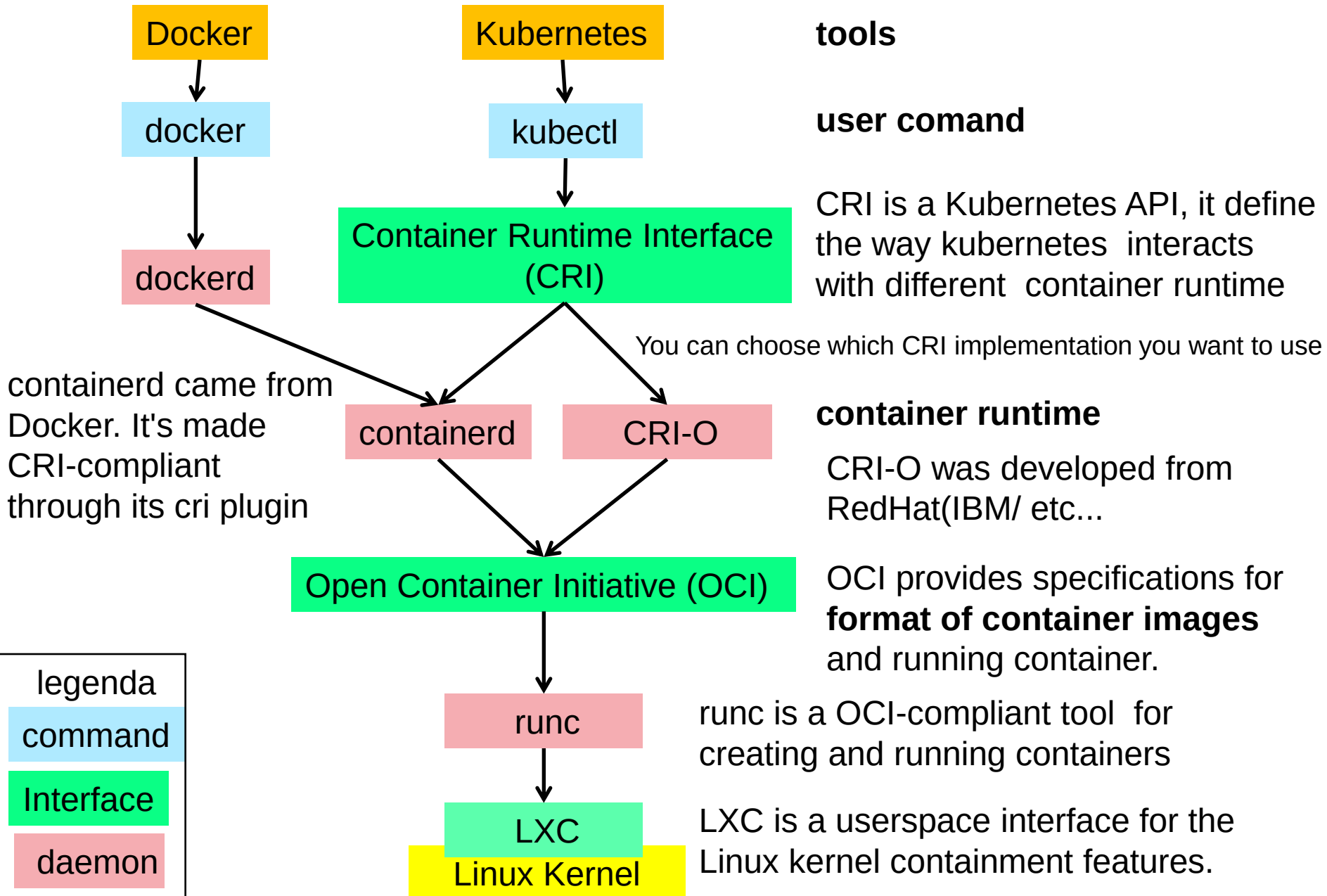
Leggere qui per informazioni (06-09-2017)

<https://docs.microsoft.com/it-it/windows-server/get-started/getting-started-with-nano-server>

Essenzialmente:

- no interfacce grafiche.
- solo eseguibili a 64 bit.
- best practices analyzer disabilitato
- consente attivazione automatica senza Product Key
- ...

Container ecosystems in Linux pre 2018



Container ecosystems in Linux today

